



Explainable AI for reinforcement learning based dynamic scheduling solutions in semiconductor manufacturing

Alessandro Immordino^{1,2} · Patrick Stöckermann^{1,3} · Niels Hayen^{1,4} · Thomas Altenmüller¹ · Gian Antonio Susto² · Martin Gebser³ · Konstantin Schekotihin³ · Georg Seidel⁵

Received: 19 February 2025 / Accepted: 21 May 2025 / Published online: 16 June 2025
© The Author(s) 2025, modified publication 2026

Abstract

The scheduling of complex job shop manufacturing environments is a difficult and NP-hard optimization problem. It is tackled increasingly with AI-based methods, often by reinforcement learning approaches. So far, this is mostly an academic endeavor as the trust in such systems is limited. The neural networks representing the policy are often seen as a black box by domain experts. By proposing a holistic approach to make such policies more interpretable and explainable, we hope to bridge the gap between academia and industry. With increasing trust in the solution, it becomes closer to being deployed in a real manufacturing environment. We propose a combination of different statistical and well-established ML-based methodologies to analyze single state-action explanations as well as the overall strategy and apply them to pretrained agents on open-source benchmark simulation models. This allows us to deliver humanly understandable explanations for the policies of the analyzed agents.

Keywords Explainable AI · Explainable RL · Semiconductor Manufacturing · Reinforcement Learning · Dispatching · Scheduling

Introduction

Scheduling and dispatching problems in semiconductor front-end manufacturing are popular topics in the literature due to their complexity and optimization potential. However, there is a huge gap regarding the evaluation and deployment of such solutions. Developing a framework able to work in a real-world semiconductor manufacturing scenario requires the commitment of several experts in different domains. Hence, all the proposed solutions have to be transparent and understandable by people with different backgrounds in order to be accepted and adopted. Neural Network (NN) based approaches are able to overcome certain limitations of state-of-the-art methods, thanks to the ability to find patterns in the used data not grasped by other approaches such as heuristics or Constraint Programming (CP). Traditional approaches are typically limited to local optimization and provide deterministic solutions, without considering important stochastic properties of the environment, such as machine failures. However, the lack of transparency regarding the results and the solution path is a challenge during the validation of NN based approaches.

Alessandro Immordino, Patrick Stöckermann and Niels Hayen authors have contributed equally to this work.

✉ Alessandro Immordino
alessandro.immordino@infineon.com

✉ Patrick Stöckermann
patrick.stoeckermann@infineon.com

✉ Niels Hayen
nielsfrederik.hayen@infineon.com

¹ Infineon Technologies AG, Am Campeon 1-15, 85579 Neubiberg, Germany

² Department of Information Engineering, University of Padua, Via Gradenigo 6/B, 35131 Padova, Italy

³ Department of Artificial Intelligence and Cybersecurity, University of Klagenfurt, Universitätsstraße 65-67, 9020 Klagenfurt am Wörthersee, Austria

⁴ School of Management, Technical University of Munich, Arcisstraße 21, 80333 München, Germany

⁵ Infineon Technologies Austria, Siemensstraße 2, 9500 Villach, Austria

In the context of semiconductor manufacturing, this problem is particularly challenging, since experts in production with no or limited Machine-Learning (ML) background have to assess the quality of the solution found. The aim of this work is to analyze the dispatching strategy found by a trained agent through Deep Reinforcement Learning (DRL), and to provide a framework capable of capturing the main insights of the learned policy. The challenge is to find an accurate representation of the learned dispatching policy, which is clear to manufacturing experts. Additionally, data scientists can benefit from an enhanced interpretability for model improvements. Trustworthiness is crucial for a constantly active and sensitive manufacturing process. However, it is not an intrinsic property of NNs (Arrieta et al., 2020), especially not in the field of DRL (Heuillet et al., 2021). Our goal is to enhance the trustworthiness by applying additional post-hoc methods to the trained model.

The main contributions of this work can be summarized as follows:

- We propose a novel combination of well-established Explainable Artificial Intelligence (XAI) methods with complementary statistical approaches to analyze the learned policy of a DRL agent from multiple perspectives.
- The method enhances the robustness of the analysis in a bottom-up fashion, moving from single state-action explanations to a comprehensive description of the overall strategy.
- We apply the method on the policy of a DRL agent which was previously trained for lot dispatching in semiconductor manufacturing and is considered as given in this work. We compare the findings for an agent trained on a small simulation model and for an agent trained on an industrial scale facility with more than 1000 tools. To the best of our knowledge, this is the first work to consider Explainable Reinforcement Learning (XRL) for lot dispatching in semiconductor manufacturing under realistic settings and industrial scale.
- We demonstrate that, despite employing different analyses at varied levels of abstraction, consistent conclusions regarding the learned policy can be derived and they are aligned with the optimization goals of the DRL agent, resulting in a coherent framework that significantly enhances trustworthiness.

Fundamentals and related work

In the following section, we introduce the problem setting of scheduling in the semiconductor frontend manufacturing domain. We then explain the basics of Reinforcement Learning (RL) and its training. Lastly, we give an overview of

explainability in the context of Artificial Intelligence (AI) and RL specifically.

Semiconductor factory scheduling problem

The Semiconductor Factory Scheduling Problem (SFSP) is a crucial optimization problem for achieving cost-efficient operations in the semiconductor industry. It is a variant of the well-known Job-Shop Scheduling Problem (JSSP) extended with additional constraints (Mönch et al., 2011, 2013). Those constraints can include dedications, meaning individual processes can only be performed on a subset of parallel machines. Furthermore, the process flow can include hundreds of steps and hundreds of machines. The processing times and transport times can be stochastic. Additional properties of such a manufacturing system are reentrant flows, meaning a product can be processed multiple times on the same machines during its process flow. Furthermore, this setup incorporates batching, meaning multiple lots can be processed together if their required process parameters match.

The conceptually simpler JSSP is already NP-complete for more than two machines (Garey et al., 1976) and therefore intractable, as are its more complex variants. The complexity is further increased in case the inherent stochasticity and constraints of the SFSP are considered (Mönch et al., 2013). It is thus often decomposed into subproblems like the Parallel Machine Scheduling Problem (PMSP) and solved with Mixed Integer Programming (MIP) (Ku & Beck, 2016) or CP approaches (Da Col & Teppan, 2022; Klemmt et al., 2017). Although these approaches can deliver very good results for the subproblem, the solution might be insufficient for the full SFSP.

In practice, semiconductor manufacturers often use simple dispatching heuristics which deliver reliable but typically suboptimal results. This approach can be further improved by use of more advanced metaheuristics. On the downside, heuristic based approaches often need to be fine tuned by domain experts (Mönch et al., 2013).

In order to directly optimize towards a global optimum regarding Key Performance Indicators (KPIs) like throughput and tardiness of the jobs, evolutionary methods like genetic algorithms can be used (Kovács et al., 2023). In contrast to static policies obtained with traditional evolutionary methods, RL approaches are able to learn adaptive policies that react to the state of the queue of waiting lots in front of the tools or the state of the fab. RL-agents can be trained to obtain an optimized dispatching policy by interaction with simulations of semiconductor frontend manufacturing facilities (Tassel et al., 2023; Stöckermann et al., 2023). This is enabled by the use of Deep Neural Networks which are able to capture complex relationships of the input data, namely the observation of the manufacturing environment.

Reinforcement learning

RL is a technique for self-learning systems that enables the optimization of a behavioral policy through interaction with complex environments (Sutton & Barto, 1998). Usually, those environments are formalized as a Markov Decision Process (MDP).

RL algorithms typically optimize a cumulative reward R over a number of decision steps T . The reward definition (Sutton & Barto, 1998)

$$R = \sum_{t=0}^{T-1} \gamma^t \cdot r_{t+1} \quad (1)$$

can include direct returns r_t for each time step t . Intermediate rewards are weighted by the discount factor γ .

Popular optimization approaches include the policy-based Proximal Policy Optimization (PPO) (Schulman et al., 2017; Mnih et al., 2016) and value-based Double Deep Q-Network (DQN) (Mnih et al., 2013). Evolution Strategies (ES)-based approaches as proposed by Salimans et al. (2017) were found to be a scalable alternative in the domain of dispatching and scheduling (Tassel et al., 2023; Stöckermann et al., 2023) as it solves the credit assignment problem (Sutton & Barto, 1998) by just calculating one reward $R = r_T$ at the end of each episode. Furthermore, the Markov property (Sutton & Barto, 1998) is not required to hold for the environment in the case of ES. In this work, we use the Covariance Matrix Adaptation (CMA) approach by Hansen and Ostermeier (2001) to improve over (Salimans et al., 2017) regarding the convergence speed by sampling the solution space while considering the covariance of the parameters θ under optimization.

Covariance Matrix Adaptation Evolution Strategies (CMA-ES) is an optimization method which aims to minimize an arbitrary objective function $F(x_1, \dots, x_i)$ that directly depends on a set of decision variables $x = [x_1, \dots, x_i]$. In this work, the set of decision variables is represented by the weights of a NN while the objective function is given by a mathematical combination of the main KPIs to be improved. The first step of the algorithm consists of creating a multi-dimensional Gaussian distribution over the NN's parameter space with the vectors mean m and covariance C defined during the NN's initialization. Afterwards, a number of samples, namely candidate solutions, each of them representing a set of NN's weights, are retrieved at each iteration t (Hansen & Ostermeier, 2001):

$$x_t = m_{t-1} + \sigma y_t, \text{ where } y_t \sim N(0, C_{t-1}). \quad (2)$$

After evaluating each candidate solution according to the fitness function, the best samples are used to update m_t and C_t for the next sampling phase. The idea behind this procedure

is to iteratively refine the distribution parameters according to the best performing candidate solutions, with the purpose to let the mean m shift to a region of the parameter space where better solutions are with a confidence given by the covariance matrix C .

Attention-mechanism

The scaled dot-product attention (Vaswani et al., 2017) enables the sequential processing of vectors as an input to the NN. While it is used in Natural Language Processing to capture relationships between words, we use it to learn relationships between lot representation vectors. The original representation of the sequentially fed vectors is projected to three new representations, namely queries, keys, and values. It is computed by applying the softmax function on the scaled dot product of the query Q with the transposed keys K (Vaswani et al., 2017):

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (3)$$

The dot product is scaled using the dimension d_k of the key projection, and the softmax result is multiplied with the value V . As only the matrices for projecting the original representation to K , Q and V are learnable parameters, this enables us to feed sequences of variable length to the mechanism without changing the structure of the NN.

Explainable artificial intelligence

XAI works as an interface that bridges the gap between complex algorithmic processes and human understanding (Guidotti et al., 2019). We refer to Arrieta et al. (2020), who collect about 400 research papers to provide an in-depth analysis of XAI methods and taxonomies, for a more detailed overview. According to them and similar taxonomies established throughout literature (Molnar, 2022; Dwivedi et al., 2023; Guidotti et al., 2019), our work falls in the category of post-hoc explainability since we mostly employ techniques used to convert an existing non-interpretable model into an explainable one.

Chen (2023) proposes the use of SHapley Additive exPlanations (SHAP) (Lundberg & Lee, 2017), a prominent model agnostic post-hoc method based on Shapley values (Shapley, 1953) and similar to LIME (Ribeiro et al., 2016), in the scheduling domain. Shapley values are a concept from cooperative game theory that calculates the payout for each member of a coalition based on their marginal contributions. It is the unique value function satisfying the properties of symmetry, efficiency, dummy and additivity (Shapley, 1953). In the context of machine learning, every member can be con-

sidered a feature of a model, while the game can be viewed as a prediction task.

Since the structure of the underlying NN uses a self-attention mechanism (Vaswani et al., 2017), useful information can be directly extrapolated from the attention weights (Xu et al., 2015). This inherent interpretability is often used to relate words in natural language processing (Vig, 2019), but can be adapted to relate jobs and their features in a manufacturing context.

Other common post-hoc methods include the generation of counterfactual explanations, which describe the smallest change of feature values possible that leads to a different prediction outcome or counterfactual state. They mostly rely on exogenous explainers, which cannot guarantee counterfactual states to be compliant with the initial feature space (Guidotti, 2024). Olson et al. (2021), i.e., use generative approaches in this context.

Relying only on post-hoc explainability could lead to misleading interpretations of the learned policy. The gap between conclusions and reality can be improved by cross-validating different methods with each other (Retzlaff et al., 2024) and ultimately checking if the explanations derived from the post-hoc analysis are aligned with the optimization goals for which the agent is trained (Kadir et al., 2023; Hickling et al., 2024). XRL covers a large spectrum of techniques that analyze an RL agent's policy on different levels. The state-action analysis extracts the main features that drive the agent's decisions, while other methods consider the learning process at policy-level in depth (Amir & Amir, 2018). The higher the level of abstraction of the analysis, the broader are the possible resulting conclusions (Milani et al., 2024). As we utilize the CMA-ES algorithm to optimize the NN's weights, we do not rely on a dense reward signal provided at every state-action transition nor a value function estimator, as seen in actor-critic DRL methods (Konda & Tsitsiklis, 1999). Consequently, this prevents the adoption of certain XAI techniques that depend on reward-decomposition or Q-function estimators (Gottesman et al., 2020; Huang et al., 2018; Bica et al., 2021; Guo et al., 2021).

Similar to the optimization approach and domain described in this work, Kuhnle et al. (2022) apply XRL for production control in a job-shop setting. The environment is similar, but substantially smaller and less complex than the biggest instance analyzed in this paper. The authors propose to explain the RL solution by comparing it with rule-based benchmarks such as simple heuristic priority rules. Furthermore, the agent's behavior is extracted with decision trees and saliency maps (Mørch et al., 1995) are used to explain specific actions. Saliency maps are often used in computer vision to highlight the areas in an image that contributed the most to the output of the NN. It is more specifically the influence on the output, in case a specific input channel is removed (Mørch et al., 1995). Kuhnle et al. (2022) use it to

highlight the most relevant areas in a visual representation of the manufacturing system for individual decisions. However, this is not possible in our case as we are not observing the entire simulator state but only the lots in an individual queue in order to achieve better scalability. However, the investigation of the attention weights described in section 4.2 gives some similar context. Kuhnle et al. (2022) conclude that the used methods are not able to fully capture the strategy of the agent but rather explain individual actions.

Analyzed Instances

In this section, we introduce the instances to which we apply our adapted methods in order to achieve better explainability. We first published the training method and experiment results for large scale optimization of a digital twin of a real manufacturing facility in Stöckermann et al. (2023) and then extended the experiments to multiple benchmark simulation models in Stöckermann et al. (2025). The resulting trained NNs of the latter publication are used in inference mode in this work to perform the explainability analysis. The open source benchmark models are well suited for the analysis, as we are not constrained by confidentiality and can go in depth regarding the features and underlying data.

Simulation Models

We conduct our explainability experiments on agents which are trained to optimize two instances of public benchmark datasets for the SFSP. The first instance is the Minifab model (El-Khouly et al., 2009; Spier & Kempf, 1995) developed by Intel and Arizona State University as a common testing ground between industry and academia. It entails 5 machines and 6 processing steps each for 3 different products. The model is ideal to test new ideas and still understand the relatively simple relations between processing steps, products, machines and the implemented dispatching strategies. Furthermore, we scale our approach to the Semiconductor Manufacturing Testbed 2020 (SMT2020) (Kopp et al., 2020) with more than 500 operations, more than 1000 machines, complex constraints and probabilistic properties such as quality checks.

At each point in time, when a tool in the simulation becomes idle, it calls the RL-agent and requests a dispatching decision, namely which lot to dispatch next. The agent then takes the entire queue of lots waiting in front of that tool as an input and assigns a score to each lot. The lots are then sorted according to that score and the one with the highest score is dispatched next. If the tool is able to batch lots together, the one with the highest score plus the next lots with the next highest scores and fitting setup requirements are dispatched together as a batch. To optimize the NN representing the pol-

icy of the agent, we use the CMA-ES algorithm (Hansen & Ostermeier, 2001) also utilized in Stöckermann et al. (2023) and Stöckermann et al. (2025), which show that this is an effective way of solving the SFSP for large scale instances.

Policy and Neural Network Architecture

The NN architecture subject to the CMA-ES optimization uses an attention mechanism and a feed-forward layer to map observations to actions (Stöckermann et al., 2025). The overall architecture for the policy is shown in Figure 1, including the pre-processing steps. One of the challenges of this system is that the queue of waiting lots can vary in length. To solve this problem, the proposed architecture uses an attention mechanism which is size agnostic and can handle any number of lots. The individual lot representations are projected to a lower dimensional space to reduce the number of required parameters. The attention mechanism allows the context of other lot representations to be injected into each lot representation independently of the number of lots.

The NN architecture includes a preprocessing step, an attention mechanism and a feed-forward layer. All non-boolean input parameters are float values, normalized using z-score normalization with running mean and standard deviation. Mean and standard deviation are calculated during training and synchronized over parallel simulations to ensure they are within an appropriate range. Batch normalization is not used as it would only consider the context of values within a single queue. The self-attention mechanism can capture dependencies or relationships between different elements in the input sequence based on their initial features (Vaswani et al., 2017). The attention weights are computed using the query (Q) and key (K) matrices, and the resulting weighted sum of the initial features provides the context representation. The attention mechanism focuses on different elements of the initial features based on their relevance determined by the attention weights. The result of the attention mechanism is projected to a lower dimension and passed through a feed-forward layer to calculate a score for each lot. Before the last feed-forward layer, the result of the attention is coupled with the original lot features in a sum operation using a skip-connection. This procedure is typical after the last self-attention projection (Vaswani et al., 2017) and it is used in our case to preserve the key properties of the original lots and it will be subject of discussion in the NN weight analysis section. The NN's attention mechanism allows it to handle variable-length input queues, making it scalable for large instances and efficient in terms of trainable parameters. The following input features are included in the observation for each individual lot:

- Time to fab due date
- Time to step due date

- Waiting time at current step
- Expected processing time
- Expected remaining cycle time
- Number of wafers in lot
- Boolean flag indicating whether the current tool is a batch tool
- Percentage of available wafers to form a full batch
- Setup time of the machine if the lot would be dispatched next

We differentiate between the due date at which the lot should finish all steps to satisfy the order's due date and the step due date for a specific processing step. The latter is based on the fab due date and calculated by subtracting the expected cycle times for the remaining processing steps.

The final policy optimized through CMA-ES minimizes the tardiness and maximize the overall throughput, as shown in Figure 2.

Methods

As each of the methods presented in the following subsections has its strengths and weaknesses, we propose to utilize them only in combination. Figure 3 shows the overall concept of how these methods can be combined to gain a holistic overview of the agent's actions, decision sensitivity, strategy and inner workings.

Feature Importance Analysis with Queue-Wise SHAP

The Shapley value $\phi_i(N, v)$ of member i is defined as

$$\phi_i(N, v) = \frac{1}{|N|!} \sum_{S \subseteq N \setminus \{i\}} |S|!(|N| - |S| - 1)! [v(S \cup \{i\}) - v(S)] \quad (4)$$

where N is the set of all members, S is the possible subset of the members and v is a function of the added value evaluated for the subset with and without the member i . Thus, the Shapley value averages the difference in value with and without the member i ($[v(S \cup \{i\}) - v(S)]$) for all possible ways to form the coalitions S prior to adding member i ($|S|!$) and all possible ways to add the remaining members ($(|N| - |S| - 1)!$) over the total number of possible ordering of all members ($|N|!$).

Many machine learning methods can however not handle sparse inputs. Thus, features cannot be left out to calculate the Shapley values for different coalitions. The added value is therefore computed by randomly replacing the features to be left out with a random sample of its value. Thus, the deviation from the average value is computed. The exact calculation of

Fig. 1 The architecture of the NN includes the self-attention mechanism. Lot representations are initially projected and subsequently fed through the attention mechanism as well as normalization and feed-forward layers. We used the same NN architecture and Figure in our previous publication (Stöckermann et al., 2023)

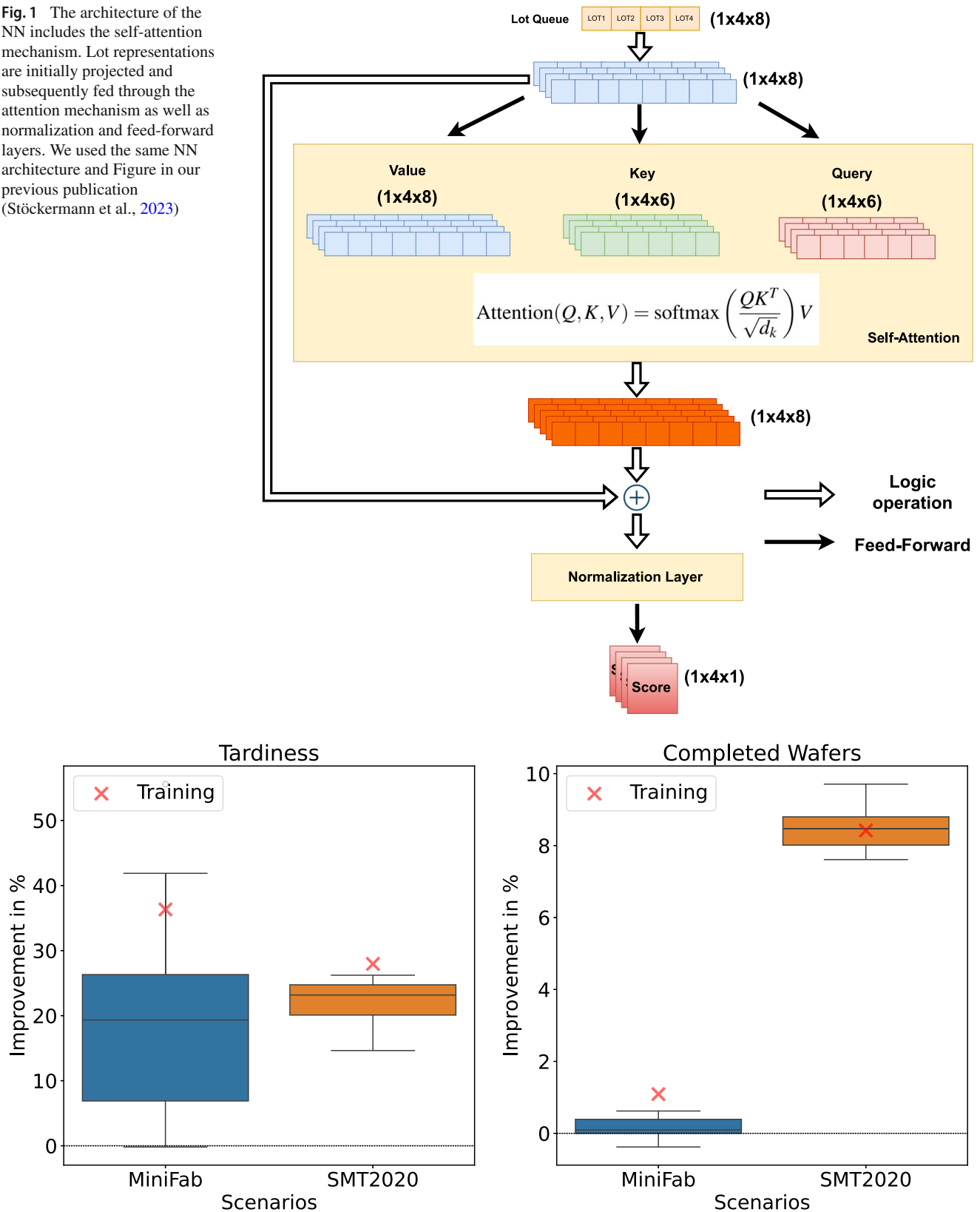


Fig. 2 Test results visualizing the improvement of the trained agent compared to the heuristic reference for the MiniFab and SMT2020 models

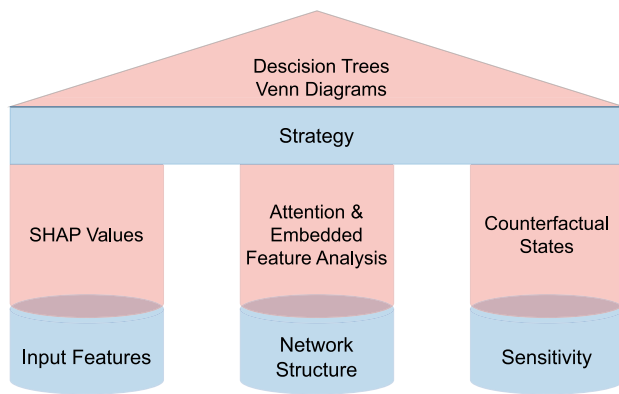


Fig. 3 Concept for holistic analysis

the Shapley value often is too computationally expensive due to the exponential increase in possible permutations. Therefore, the value is usually approximated. This can be done with Monte-Carlo sampling (Strumbelj & Kononenko, 2014):

$$\hat{\phi}_i(M, \hat{v}) = \frac{1}{M} \sum_{m=1}^M (\hat{v}(x_{+i}^m) - \hat{v}(x_{-i}^m)). \quad (5)$$

For this, a random instance z and a random permutation of the feature order are sampled M times. Then, feature values for features after feature i in the feature order are replaced by features from z to form x_{+i}^m . Instead, x_{-i}^m is formed by replacing feature i with a value from the random instance z in addition to replacing the same features as for x_{+i}^m . This way, the difference compared to the average value with and without feature i can be approximated, using a random order, to randomly generate different coalitions. The Shapley value is then calculated as the average over these marginal contributions. Antithetic sampling can be used to reduce variance for Monte-Carlo methods and in this context refers to using random permutations in their original and additionally their reversed order (Staudacher & Pollmann, 2023).

We use the *Permutation Explainer* provided in the python package *shap* (Lundberg & Lee, 2017) to perform this calculation using antithetic sampling. Because the presented model architecture accounts for different input sizes using the attention mechanism, we expect the model output to depend on the input size used. Therefore, we adapt the calculation to be performed queue-wise. We calculate the Shapley values for each lot while respecting the original input size regarding the respective queue length and sample only from lots within the same queue. The method used for the calculation is summarized in Algorithm 1 and is an adapted version of the algorithm proposed by Lundberg and Lee (2017).

Algorithm 1 Queue-wise permutation SHAP

Input: Number of permutations M , set of queues $\mathcal{Q}$

for queue q in $\mathcal{Q}$ **do**

for lot l in q **do**

 Determine indices list I of varying input features

for $m = 1, \dots, M$ **do**

 Shuffle indices

for feature f in I **do**

 Set input features of lot l up to position of f in I to original values of l

 Set input features of lot l after position of f once in- and once excluding f in I to randomly sampled values from other lots in q

 Compute difference $\delta = \hat{v}(x_{+f}^m) - \hat{v}(x_{-f}^m)$ and antithetic difference $\hat{\delta} = \hat{v}(\hat{x}_{+f}^m) - \hat{v}(\hat{x}_{-f}^m)$

 Add differences to lot SHAP value for f $\hat{\phi}_{lf}^m \leftarrow \hat{\phi}_{lf}^{m-1} + \delta + \hat{\delta}$

end for

 Divide cumulated SHAP by $2M$ $\hat{\phi}_{lf}^M \leftarrow \frac{1}{2M} \hat{\phi}_{lf}^M$

 Set queue score average with original input values as expected value

end for

 Add SHAP values for l to aggregated SHAP values for q

end for

Network Analysis

To better understand the inner processing of the NN (see Figure 1), we evaluate intermediate outputs from layers within the network. We analyze two specific steps: the result of the softmax function before multiplying it with the value projection, which, due to its dimensionality, we call 'lot on lot attention', as well as the values after the final layer before summation to the final score, which we call 'embedded feature contributions'. We select those two steps as it gives us two distinct explanations: the overall contributions of the input features on the final score of individual lots as well as their impact on the features of other lots.

The lot on lot attention is a matrix (see Figure 8 in the results section) with both dimensions corresponding to the input queue's length. Because of its multiplication in the forward propagation, it can be interpreted as the influence of the column lots on the feature values of the row lots. Therefore, lots with higher values in their columns have more impact on other lots, while lots with high values in their rows are more highly impacted by other lots in the queue.

The embedded feature contributions are a vector of the same dimensionality as a single lot's input vector. Because there is only one normalization and one linear layer following the summation of skip connection and self-attention values, each value contains elements of its original input

feature through the skip connection as well as from the attention mechanism, which are directly added to the final score. Hence, the embedded feature contribution captures the “direct impact” of its corresponding input feature altered by the “context” of the attention output on the final score. A higher variation of the embedded score contributions thus indicates a stronger influence of the embedded feature on the final score difference between lots and thus decision impact. If a contribution is small compared to the features impact determined in other methods, this feature must influence the decision-making process mainly via the attention mechanism. The visualization of these relations can be seen in the results chapter (Figure 8).

Sensitivity Analysis

To determine decision robustness, we use counterfactual explanations to analyze the sensitivity of the dispatching decision to changes of the input features of the picked lot. This helps us to not only deduct a measure for robustness, but also a measure for feature importance, as features leading to more frequent changes of the decision are deemed more important than others.

Using a generative approach as by Olson et al. (2021) is limited in the manufacturing use case, as this complex environment has many restrictions regarding the possible values which the input features can assume on each simulation day. Moreover, solely determining counterfactual states with respect to some minimum distance, as done by most approaches, hinders comparability with other methods proposed in our holistic approach. We therefore employ a custom algorithm as a statistical method, which is illustrated in Algorithm 2. For each queue representing the input state, we iteratively change the value of each input feature for the lot with the highest score in small steps s and observe if this leads to another lot being chosen. The goal of this procedure is to find out how much each input feature has to be tweaked to induce a different decision and if it is changing the decision at all. To ensure realistic feature combinations, we only sample within each queue.

One step s consists therefore in replacing the top scored lot's features with the next highest or lowest value among the possible values that the corresponding feature can take within that queue. We record the percentage of queues for which we trigger a change of decision for each feature. Furthermore, we report the distribution of how many incremental steps it took and in which direction for the cases that led to a different decision.

Strategy Analysis

As proposed by Kuhnle et al. (2022), we approximate the agent's behavior using decision trees. For easier interpretabil-

Algorithm 2 Counterfactual State Analysis

Input: Max number of steps S , set of queues Q
 List $L_{f+} = []$ and $L_{f-} = []$ for each feature f
 Number of induced changes $c_f = 0$ for each feature f

for queue q in Q **do**
 for feature f in lot l_0 with highest score **do**
 Get list all values V for f in q
 Sort V
 Get list V_+ and V_- of values that are higher resp. lower than v_0 of l_0
 Step $s = 0$
 Change $c = \text{False}$
 for value v in V_+ **do**
 $s = s + 1$
 Modify feature f of lot l_0 to v
 Compute decision of agent
 if agent's decision is not l_0 **then**
 Append s to L_{f+}
 $c = \text{True}$
 end for
 Step $s = 0$
 for value v in V_- **do**
 $s = s + 1$
 Modify feature f of lot l_0 to v
 Compute decision of agent
 if agent's decision is not l_0 **then**
 Append s to L_{f-}
 $c = \text{True}$
 break
 end for
 if c is True **then**
 $c_f = c_f + 1$
 end for
end for

Output: For each feature f : Percentage of queues q that were able to induce a change $c_f / \text{len}(Q)$, list of steps required for each queue per direction L_{f+} , L_{f-}

ity, we use a classifier tree to classify lots in 'dispatched' and 'not dispatched' instead of predicting a score. Because the agent makes a decision based on the entire queue, we need to capture the relationship between the features of a lot and the other lots in the queue. Therefore, we introduce 3 additional features for each original feature: two binary features that state if the lot has the maximum or minimum value of this feature within the queue as well as one feature for its percentile w.r.t. the rest of the queue. Additionally, we add the queue length as a feature. The tree is trained to predict if a lot would be dispatched by the agent or not just with the original features of the lot and the just mentioned additional features as an input.

We use the *DecisionTreeClassifier* from scikit-learn (Pedregosa et al., 2011) and deduct a grid search-based hyperparameter tuning to achieve a high F1-Score. We vary the tree's information gain criterion, which determines the binary splits at each node, between gini and entropy. The tree depth is explored from 3 to 7 layers to ensure the readability of the tree. We limit the number of features considered for each split and train with 10 different random seeds to reduce the risk of

reaching a local minimum. The limit is varied between the number n_f of input features and $\frac{n_f}{4}$ in 4 equally distanced steps. We use 85 % of the dataset as training data and 15 % as a test set to determine generalization performance of the tree. We choose the best performing trees with regard to their F1-Score for the test set as it provides a balanced measure of precision and recall.

Methodological Strengths and Weaknesses

In this subsection, we explain how the methods complement each other considering their strengths and weaknesses. The main properties of the methods are summarized in the following section.

The **SHAP Values** are a popular tool to estimate the impact of individual features on the output values of NNs, namely the scores of the lots. Compared to the similar popular method LIME (Ribeiro et al., 2016), SHAP offers additional theoretical properties of Shapley values and is closer to human intuition in user studies (Lundberg & Lee, 2017). One disadvantage is that the method only considers the impact on single lot scores, but in our use-case, the perturbations of the method change the scores of the other lots in the queue too. This is due to the attention mechanism. Furthermore, correlation among the input features is not considered in this method which is a problem for correlated features (Aas et al., 2021). Additionally, the perturbations and thus combinations of input features can be unrealistic.

The **Embedded Features** are a good tool to analyze the overall inner workings of the model, while the attention visualization is a good tool to examine these inner workings for individual decisions. However, only relatively short queues can be visualized. Compared to similar methods like in Xu et al. (2015) or Vig (2019), which offer a more detailed visual analysis, we focus on the analysis of embedded features that are closely related to the final score values and circumvent the drawback of scalability to longer queues. The analysis is based on historical data without independently varying features, but there is strong feature correlation, which however does not imply causation. Due to a required understanding of the underlying mechanisms they are more useful to data scientists.

Decision Trees split the dataset without considering the scores. The problem is redefined as a binary decision problem, classifying lots as 'dispatched' or 'not dispatched' individually for better interpretability. Due to their low representation power, decision trees can only offer limited insight into the agent's policy. They are a simplified representation of the agent's policy and may not capture all of the nuances of its behavior. This method also does not consider individually varying features. However, this has the advantage that all input data is realistic and no infeasible states are generated. Due to their resemblance of common

rule-based decision-making, they are particularly useful for domain experts. Compared to direct rule extraction, decision trees are better visually interpretable, but harder to represent for complex problems.

Counterfactual States use a naive variation of only the picked lot's features of each queue. It is a very direct measure to determine the sensitivity of each feature regarding the decision 'dispatched' or 'not dispatched'. However, it is difficult to compare the features among each other as they have different distributions. The results therefore heavily depend on the step size of the adjustment steps. The design of the steps can furthermore lead to unrealistic feature value combinations, which we limit by sampling within the current queue of each observation. While other approaches like Olson et al. (2021) avoid these problems with more sophisticated generative approaches, these can't guarantee compliance with the initial feature space at all and don't allow to deduct a direct feature-related sensitivity measure.

The different perspectives of the methods are summarized in Table 1. Besides the technical details, the methods vary in their target audience. While the network analysis is only useful for data scientist which understand the inner workings of the NN, the decision tree helps domain experts to relate the agent's strategy to well known heuristics in their domain. The SHAP analysis and counterfactual explanations are working at the edge between the two disciplines and can be useful for both parties.

The holistic combination of these methods yields two main advantages: On one hand, we can address different perspectives for relevant target audiences along the decision-making process from the importance of input features, over processing within the network to induced decision changes and the analysis of the overall strategy. On the other hand, the combination of the particular methods allows for an effective cross-validation and thus increased trustworthiness of the approach, as the methods do not share the same weaknesses. For example, we can observe if changes of features with high SHAP values lead to changed decisions with the counterfactual explanations, observe if these features explain many dispatching decisions that occur in historical data with decision trees while observing possible structural explanations for differences between the methods in the network structure analysis. When using simpler approaches, we might classify features as important that do not actually change decisions, i.e., due to other lot scores being changed, too. We might also focus on unrealistic feature combinations that do not occur in the actual data or alternatively limit our view to historical data without considering potential deviations, mistaking correlation for causation.

Table 1 Selected XAI methods and their properties

		SHAP	Network Analysis	Counterfactual Explanations	Decision Trees
Target Audience	Domain Experts	✓		✓	✓
	Data Scientists	✓	✓	✓	
Data Basis	Modified Data	✓		✓	
	Historical Data		✓		✓
Evaluation Basis	Score	✓	✓		
	Decision			✓	✓

Results: In-Depth Analysis of Presented Instances

In the following section, we apply the presented methods to the benchmark instances MiniFab and SMT2020.

Analysis of Input Features

First, we analyze the feature importance to understand which inputs are most relevant for the lot score. We conduct the SHAP analysis separately per tool, as each tool has different constraints such as batching or setup. In the following, we will focus on the lithography tools of both models to ensure comparability. The comparison of all tools would take too much space in this paper.

The SHAP values in Figure 4 for the lithography area of the MiniFab show that the time to step due date is the most important feature and biggest discriminator. This fits to the decreased tardiness in Figure 2. As expected, the setup time is weighted highly as well. The waiting time is ranked quite low, which means that the behavior is far from First-In First-Out (FIFO).

Some features, such as the ones related to batching, are not relevant for lithography and therefore have no impact. Other features could potentially be relevant, but the SHAP plot indicates that there is no visible impact. Based on these results, such features can be removed from the input in further experiments in order to reduce the number of trainable parameters of the NN and achieve faster convergence. The results for the SMT2020 model in Figure 5 are comparable and only individual features are ranked higher or lower but with the same direction. The number of tools that can process each current lot becomes relevant for this model, however without high significance.

Analysis of Decision Sensitivity

The robustness of the decisions and sensitivity of the features for the MiniFab can be evaluated by looking at the counterfactual state analysis in Figure 6. We visualize the smallest change per feature which is required for the lot with the high-

est score to lose its position as described in Algorithm 2. The adjustment steps are representing a step through the values that occur in the respective current queue.

The most sensitive feature with the highest percentage of decision changes seems to be the time to step due date which fits to the SHAP values (see Figure 4). The features remaining cycle time and waiting time are not leading to a change as often and require more adjustment steps. Most notably, the remaining cycle time has a low impact in this analysis while having high SHAP values. This could mean that it impacts, through the attention mechanism, all features in a similar direction. As an example, by increasing the remaining cycle time for the first lot we possibly increase the score of all lots and therefore do not change the decision but only the score and SHAP value. Using this method therefore helps to visualize the real impact on the decision, not only on the score as the SHAP values.

For SMT2020 (see Figure 7), we can observe similar behavior. The distribution is smoother as we have more diverse data, i.e., more possible values for the processing time. As the queues are longer, the potential adjustment steps are also more. Since there are more lots to choose per queue, the percentage of queues with decision changes is also higher. One major difference is the remaining cycle time, which has a clear direction for the SMT2020 model, however not for the MiniFab. It could be an artifact that was learned from correlations in the MiniFab model. The changes for the setup time could be low due to the lower number of lots with matching setup requirements due to the more diverse dataset.

Analysis of Network Structure

In order to understand the inner workings of the model better, we plot the correlation of the embedded features of the last layer of the NN with the final score on the left side in Figure 8. It is most notable that some features seem to have almost no impact on the final score. By correlating the most impactful embedded features with the original features, we can understand how the features interact and lead to the final score. Through this analysis, we realize, that the remaining cycle time, which is passed directly to feature 6 through the

Fig. 4 SHAP values for the lithography tools of the MiniFab model. The color indicates the magnitude of the feature value for a lot within the queue and the x-axis the impact on the model output. A single point represents one feature of one lot. A blue point with a high x-value states that this feature of the considered lot is lower than the average of other lots (color) in the queue and led to a higher score of the lot (SHAP value)

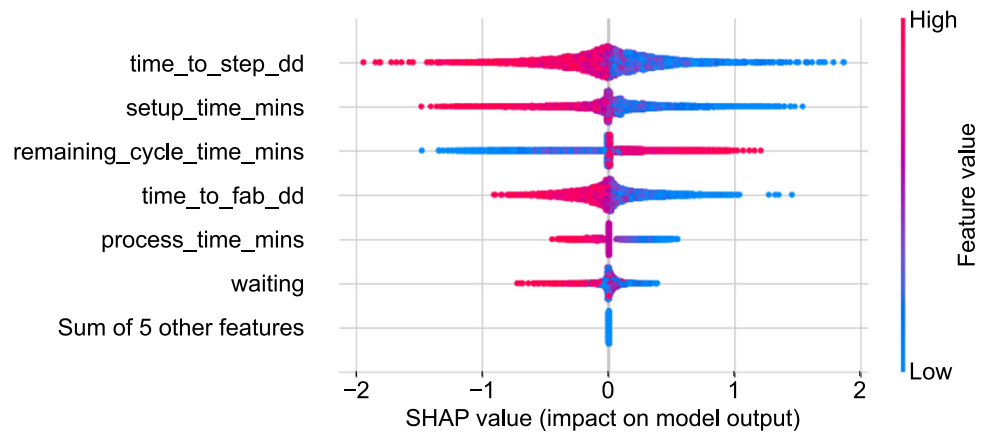


Fig. 5 SHAP values for the lithography tools of the SMT2020 model. The color indicates the magnitude of the feature value for a lot within the queue and the x-axis the impact on the model output. A single point represents one feature of one lot. A blue point with a high x-value states that this feature of the considered lot is lower than the average of other lots (color) in the queue and led to a higher score of the lot (SHAP value)

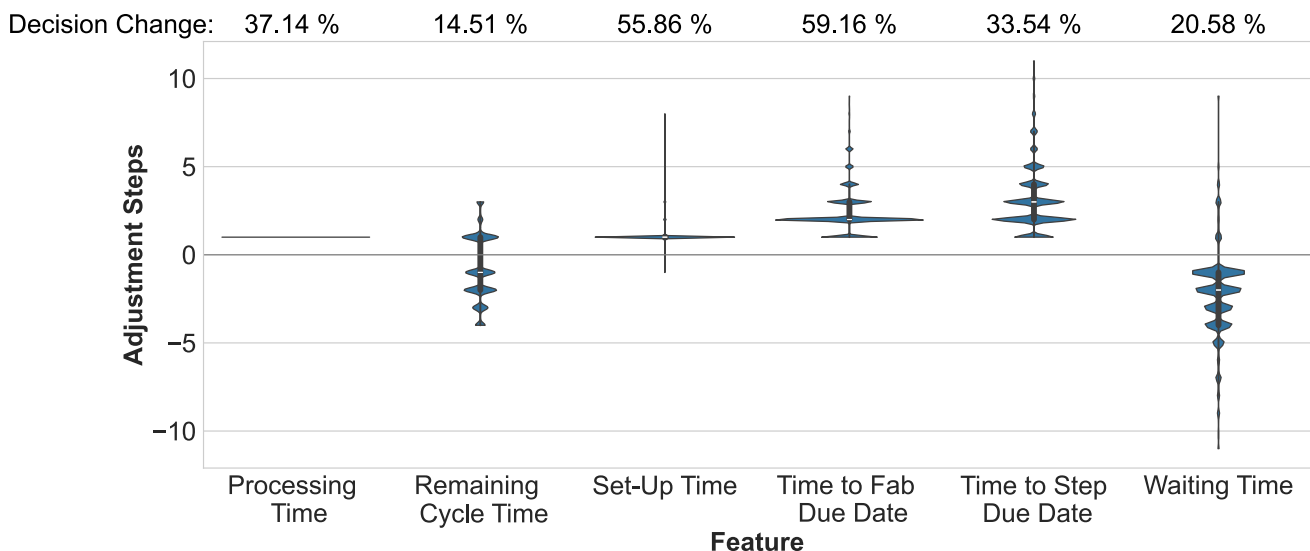
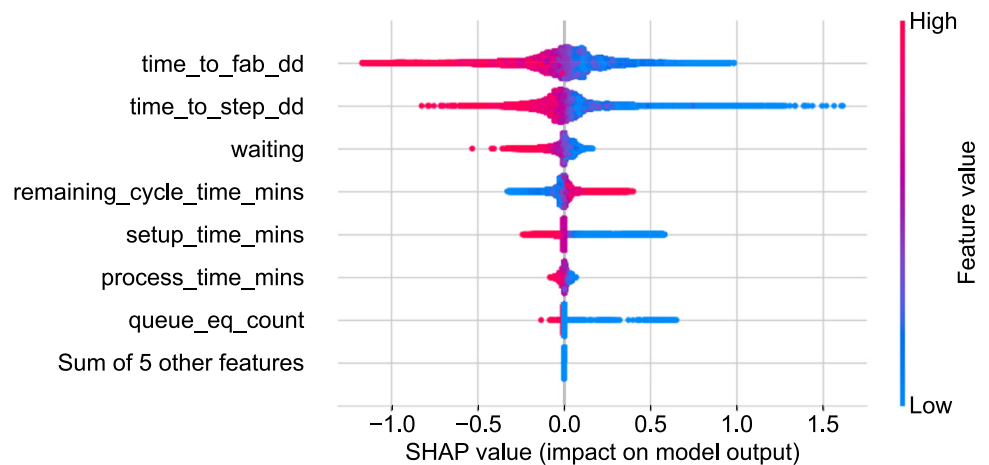


Fig. 6 Counterfactual states analysis for the MiniFab. The decision change indicates the percentage of queues for which we can induce a different decision by modifying the respective feature. The adjustment

steps represent the number of steps that was required for the change. One step is a step to the next highest (positive step) or lowest value (negative step) for the respective feature in the current queue

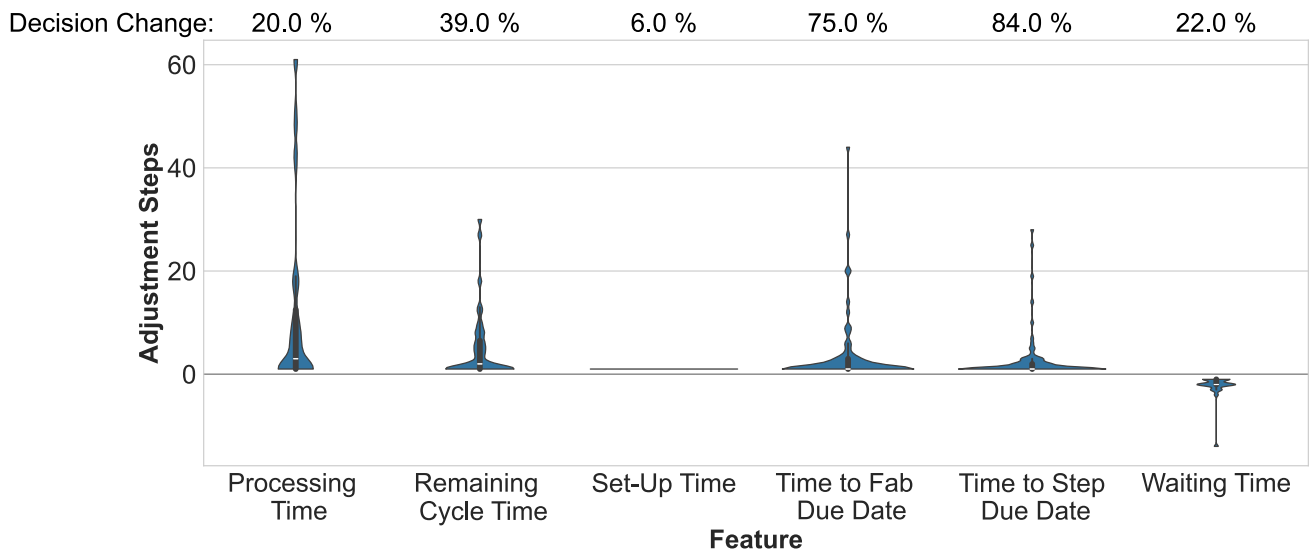


Fig. 7 Counterfactual states analysis for SMT2020. The decision change indicates the percentage of queues for which we can induce a different decision by modifying the respective feature. The adjustment

steps represent the number of steps that was required for the change. One step is a step to the next highest (positive step) or lowest value (negative step) for the respective feature in the current queue

skip connection, has a big impact on the score according to the SHAP analysis (see Figure 4), but only through the attention mechanism, as feature 6 does not significantly contribute to the score. The counterfactual states analysis visualized in Figure 6 shows that we only trigger a decision change in 14.51 % of the cases for this feature, confirming this analysis.

When comparing all 3 methods, it seems probable that this feature impacts the other lots in the queue in a similar way and not only the lot with the high remaining cycle time value. It is increasing the score of all lots and is therefore not as big of a discriminator as it seems in the SHAP analysis.

On the right side of Figure 8, the impact of each lot on the other lots through the attention mechanism is visualized as an example for one queue. This analysis is however limited to individual queues and to short ones, as the size of the matrix will be too big to visualize at some point. It is a useful tool to understand individual decisions, however not the overall behavior and strategy. In this case, lot 10 had a high impact on itself and the other lots, especially lot 5. This is due to outlier values regarding individual features, in this case a very low remaining cycle time as the lot is at its last step of the route.

Analysis of Strategy

In order to gain high level insights and compare the agent to heuristics, we train decision trees. Furthermore, we compare the strategy of the agent to simple heuristics in Venn diagrams.

Decision Tree

When training a decision tree on observations from the MiniFab model, we achieve acceptable accuracy on unseen test data (see Table 2, left) by doing hyperparameter turning regarding the inputs, tree depth, randomness and the criterion for splitting. We limit the depth to 7. The rebalancing rate refers to the factor by which we artificially increase the number of samples with label 'dispatched' in the training dataset. A rebalancing rate of 1 is thus not changing the original dataset. The trees confirm the findings above as the first splits are related to features, i.e., with high SHAP values and a high percentage of decision change in the counterfactual state analysis. Furthermore, we can derive some heuristics like "lots are not dispatched first if they have the highest processing time in the queue and the time to step due date is below 53 % of the current queue."

The resulting tree is hard to visualize, as it is quite large, even for this small model. However, Figure 9 shows a partial view of the best performing tree for lithography, which illustrates the principle.

For the SMT2020 model, we achieve less satisfactory results regarding the accuracy of the trained tree (see Table 2, right). We assume this is due to the higher diversity of the feature values. As an example, the feature processing time only has 3 different values for some of the tools in the MiniFab model. Similar to the MiniFab, the splitting order confirms the other methods, like counterfactual states or the SHAP values. However, general heuristics are difficult to derive due to the low performance of the tree.

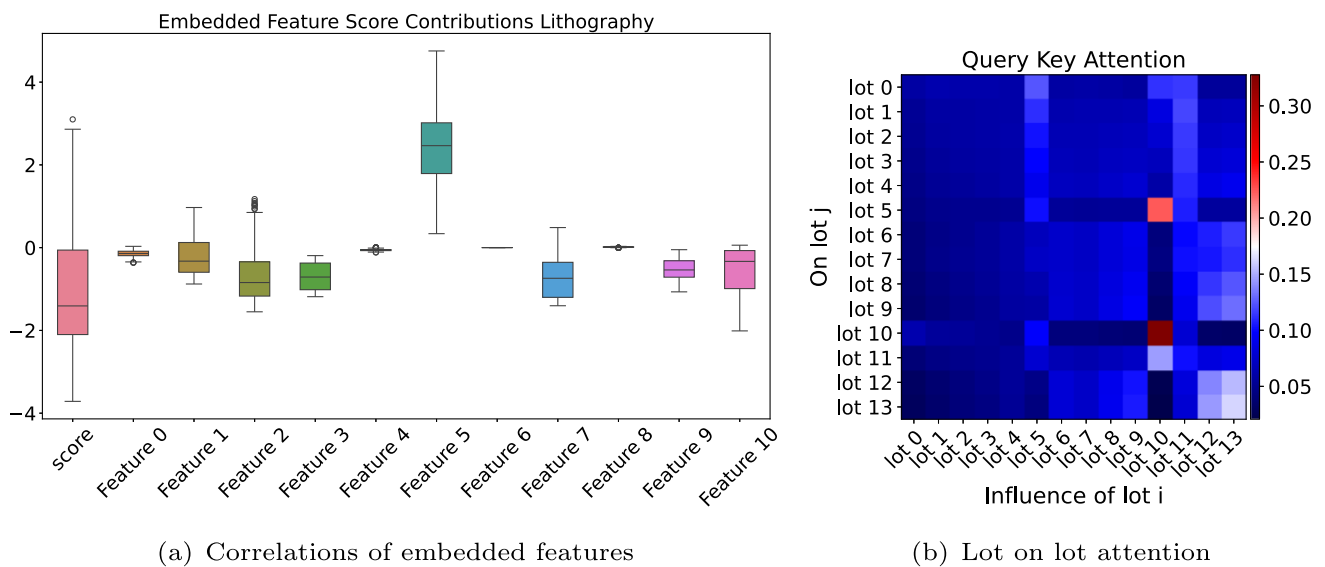


Fig. 8 Left: Embedded feature values for the MiniFab model representing the contributions of the intermediate feature values on the final score. Right: Lot on lot attention visualizing the impact each lot in the

exemplary visualized queue has on the other lots in the queue through the attention mechanism for the MiniFab

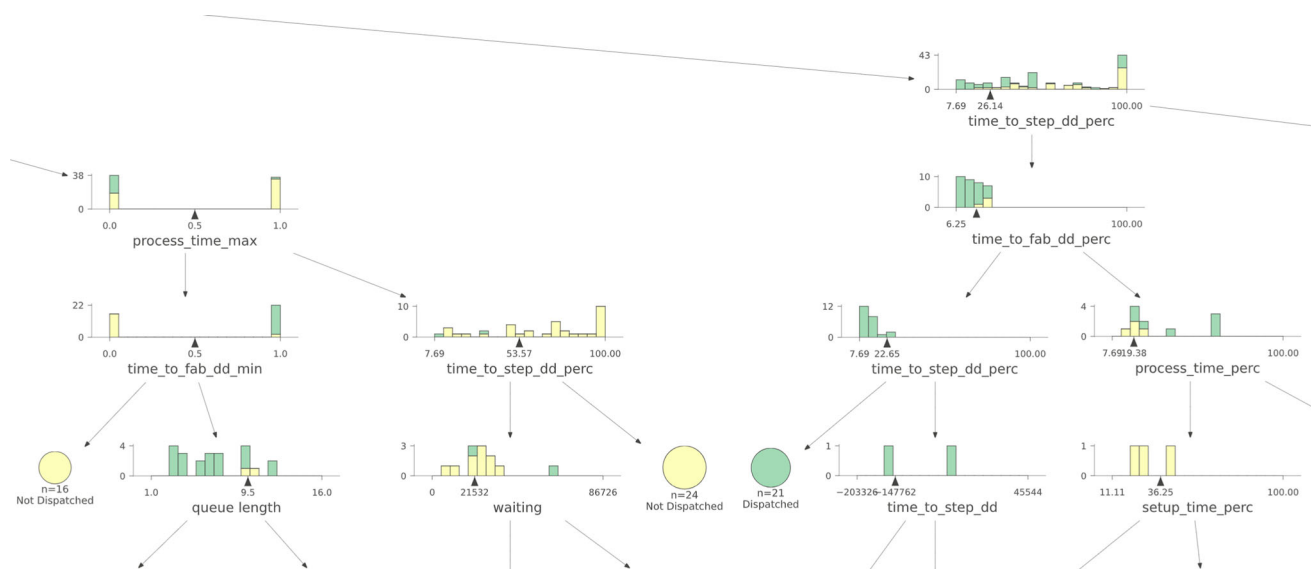


Fig. 9 Part of the decision tree trained on the MiniFab model for the lithography tools. The histograms show the feature distribution at each split, with the split value indicated by the triangle. Green areas mark dispatched lots, whereas yellow areas show not dispatched lots

For both models, we do not observe significantly better results with unlimited tree depth. Furthermore, a too big tree does not offer any practical use as it does not help much regarding human understanding.

Comparison with Default Heuristics

In order to compare the strategy of the agent with default heuristics, we visualize the overlap between the heuristics and the agent's strategy in a similar way as Kuhnle et al.

(2022). The overlap in the Venn diagram for the MiniFab (see Figure 10) shows the percentage of dispatched lots that are compliant with the respective heuristic regarding the lot with the highest score. If the heuristic is ambiguous, meaning there are, i.e., multiple lots with the same setup time, we compare if the lot chosen by the agent is also among the lots with the highest rank for the respective heuristic. Thus, a high compliance does not mean the heuristic would have chosen the same lot, it depends on the tie breaking rule for ambiguous cases. The SRPT rule is shown besides the stan-

Table 2 Test results on unseen data and hyperparameters for the decision tree with the best settings per tool trained on the observations of the MiniFab model (top) and SMT2020 model (bottom)

	Lithography	Diffusion	Implantation
Rebalancing Rate	2	1	1
Criterion	gini	gini	entropy
Tree Depth	7	7	5
Max. Split Features	6	15	20
Accuracy	0.9386	0.9689	0.9057
Balanced Accuracy	0.9012	0.9290	0.8822
Precision	0.8494	0.8286	0.8092
Recall	0.8393	0.8788	0.8333
F1-Score	0.8443	0.8529	0.8211
	Lithography	Diffusion	Implantation
Rebalancing Rate	2	1	1
Criterion	entropy	entropy	gini
Tree Depth	7	7	7
Max. Split Features	21	28	18
Accuracy	0.9712	0.9706	0.9350
Balanced Accuracy	0.8052	0.8039	0.7421
Precision	0.7505	0.5869	0.6223
Recall	0.6212	0.6242	0.5116
F1-Score	0.6797	0.6049	0.5616

dard EDD, FIFO and setup rule as it is the best performing basic individual heuristic regarding the tardiness.

As expected from the previous analyses, the biggest overlaps can be seen between the agent and the SRPT, Setup and EDD heuristics. Especially for setup, the overlap is almost 100 % as the values for the setup time are equal for many lots in the queue.

For the SMT2020 model (see Figure 11), the overlap is much smaller as the model is more complex and the queues are longer, which reduces the chance of the first lot overlapping. The only significant overlap besides the setup rule can be observed for the EDD and CR rules. The CR rule is the best performing heuristic we tested for comparison on the SMT2020 model.

Limitations

The scalability of the solution has been proven by applying it to instances of vastly different size including stochasticity, namely the MiniFab model and the SMT2020 dataset. However, especially for the bigger dataset, SMT2020, we had to adjust the implementation with computation time in mind. As some methods take hours, we parallelized them and ran the code on a compute farm to reduce the computation time

from hours to minutes. The computational time after parallelization with 64 parallel CPU cores is visualized in Table 3.

The insights get more and more difficult to interpret for increasing complexity, especially for analysis on the strategy level, such as the decision trees. The methods also depend on the structure of the NN and are particularly useful for self-attention based architectures. The network analysis would have to be entirely adapted in case a vastly different structure is used. The SHAP values, counterfactual states and decision trees are more general and do not directly depend on the architecture. However, they would have to be adapted to the observation space in case it is changed. In addition to this, singular methodological limitations as discussed in Section 4.5 can be mitigated by combining them but cannot be solved entirely.

In case of more complex resource constraints, the RL agent might have to be enhanced by combining it with a traditional scheduling method like MIP or CP, which have shown to be efficient in local scheduling problems, but limited regarding the global view and would require different XAI methods. A more complex behavior by the RL agent itself to solve such more complex constraints can only be analyzed efficiently with the given methods in an abstract manner, as most methods focus on global feature importance, statistical properties or similar. While the approach allows an investigation of single decisions in a more detailed fashion, this would require a time-consuming analysis for such complex scheduling situations.

Conclusion of Analysis

The agent has to consider three main objectives: ensure on-time delivery and efficiently handle setup and batching constraints. The agent for the MiniFab model does this by highly weighting the tardiness related metrics and balancing it with the input features related to the constraints. Namely, the average batch size is increased and the number of setup changes is lowered. These features have a high impact throughout our methods, whereas other features such as remaining cycle time only impact the decision context through the attention mechanism. The SMT2020 model additionally increased the throughput compared to the benchmark heuristics by efficient setup and batching. The agent for the SMT2020 model has a higher degree of freedom as the queues of lots are longer and there are more tools.

The high importance of setup time, batch efficiency and tardiness as input features is not surprising to manufacturing engineers in the domain. It rather validates the strategy of the agent by reassuring that the agent has found a strategy that does not include weighting features in a very counterintuitive way. That would be a warning signal which could indicate reward hacking and missing constraints. However, the exact fine tuning of these features is the strength of the method and

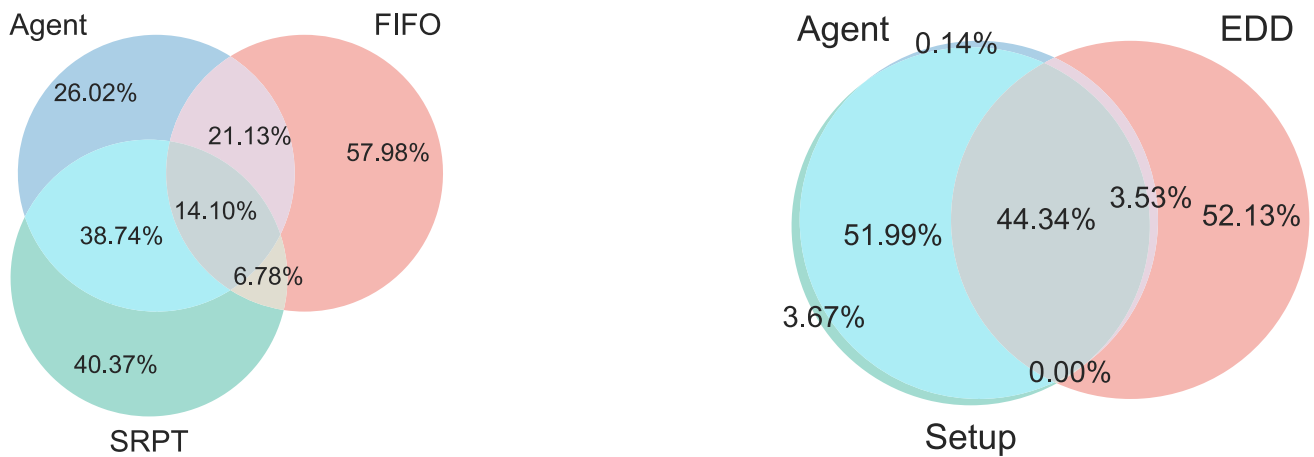


Fig. 10 Venn diagrams showcasing the compliance of the agent's strategy with default dispatching heuristics for lithography for the MiniFab

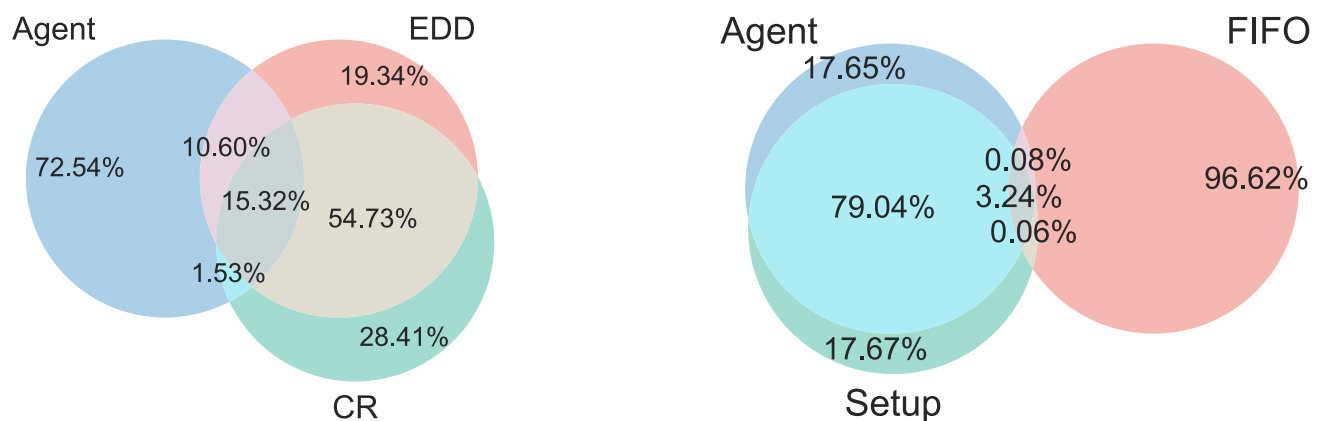


Fig. 11 Venn diagrams showcasing the compliance of the agent's strategy with default dispatching heuristics for lithography for the SMT2020 model

Table 3 Computation time for training and XAI methods for the MiniFab and SMT2020 simulation model.

	Training	SHAP	Network Analysis	Counterfactual Explanations	Decision Trees
MiniFab	Hours*	Minutes*	Seconds	Minutes	Seconds*
SMT2020	Days*	Hours*	Minutes	Hours	Minutes*

*Parallelized on 64 CPU cores

more powerful than reducing it back to a hierarchical rule based approach.

Outlook and Conclusion

We present a holistic approach to make attention-based NNs more explainable in the domain of dispatching and scheduling. It is based on the combination of multiple methods with different advantages and disadvantages. We apply these methods on a RL-agent which was trained on two benchmark instances for the SFSP and draw conclusions about its behavior and strategy. The two models have a significantly

different level of complexity and the obtained results show that some methods like decision trees cannot handle high complexity. It is especially hard to find non-trivial insights for complex instances. Simple behavioral trends can however be observed across methods, cross-validating these insights. In future work, we plan to present an inherently explainable approach to the problem, not requiring additional tools to analyze black box-like NNs to understand the policy. Other future research could focus on generalizing a similar holistic framework to other domains and network architectures, potentially expanding to a different set of XAI methods.

Acknowledgements This work was partially funded by the German Federal Ministry for Economic Affairs and Climate Action (BMWK)

project 13IK033 (SmartMan) as well as the Austrian Research Promotion Agency (FFG) project 894072 (SwarmIn) and FO999910235 (SAELING). This work was furthermore partially carried out within the MICS (Made in Italy - Circular and Sustainable) Extended Partnership and received funding from Next-GenerationEU (Italian PNRR - M4C2, Invest 1.3 - D.D. 1551.11-10-2022, PE00000004). Moreover this study was also partially carried out within the PNRR research activities of the consortium iNEST (Interconnected North-Est Innovation Ecosystem) funded by the European Union Next-GenerationEU (Piano Nazionale di Ripresa e Resilienza (PNRR) - Missione 4 Componente 2, Investimento 1.5 - D.D. 1058 23/06/2022, ECS00000043). Moreover, we acknowledge financial support under the National Recovery and Resilience Plan (NRRP), Mission 4, Component 2, Investment 1.1, "Progetti di ricerca di Rilevante Interesse Nazionale - PRIN" Call for tender No. 104 published on 2.2.2022 by the Italian Ministry of University and Research (MUR), funded by the European Union - NextGenerationEU - Project Title "Smart Monitoring and Production Optimization for Zero-waste Semiconductor Manufacturing: SMART-IC" - CUP C53D23003670006.

Funding Open access funding provided by University of Klagenfurt.

Data Availability Due to commercial reasons, new supporting data is not available. However, the original dataset of the SMT2020 model (Kopp et al., 2020) and descriptions for the MiniFab model (El-Khouly et al., 2009; Spier & Kempf, 1995) are already available.

Declarations

Conflicts of Interest The authors declare that they have no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Aas, K., Jullum, M., & Løland, A. (2021). Explaining individual predictions when features are dependent: More accurate approximations to shapley values. *Artificial Intelligence*, 298, Article 103502. <https://doi.org/10.1016/J.ARTINT.2021.103502>
- Amir, D., & Amir, O. (2018). HIGHLIGHTS: summarizing agent behavior to people. *AAMAS* (pp. 1168–1176). SC, USA / ACM: International Foundation for Autonomous Agents and Multiagent Systems Richland.
- Arrieta, A. B., Rodríguez, N. D., Ser, J. D., et al. (2020). Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information fusion*, 58, 82–115. <https://doi.org/10.1016/J.INFFUS.2019.12.012>
- Bica, I., Jarrett, D., Hüyük, A., et al. (2021). Learning "what-if" explanations for sequential decision-making. In: *ICLR*. OpenReview.net
- Chen, T. C. T. (2023). Explainable Artificial Intelligence (XAI) in Manufacturing: Methodology, Tools, and Applications. *Springer International Publishing*. <https://doi.org/10.1007/978-3-031-27961-4>
- Da Col, G., & Teppan, E. C. (2022). Industrial-size job shop scheduling with constraint programming. *Operations Research Perspectives*, 9, Article 100249. <https://doi.org/10.1016/j.orp.2022.100249>
- Dwivedi, R., Dave, D., Naik, H., et al. (2023). Explainable AI (XAI): core ideas, techniques, and solutions. *ACM Computing Surveys*, 55(9), 194:1–194:33. <https://doi.org/10.1145/3561048>
- El-Khouly, I. A., El-Kilany, K. S., & El-Sayed, A. E. (2009). Modelling and simulation of re-entrant flow shop scheduling: An application in semiconductor manufacturing. In: *2009 International Conference on Computers & Industrial Engineering*. IEEE, pp 211–216, <https://doi.org/10.1109/ICCIE.2009.5223754>
- Garey, M. R., Johnson, D. S., & Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of operations research*, 1(2), 117–129. <https://doi.org/10.1287/MOOR.1.2.117>
- Gottesman, O., Futoma, J., Liu, Y., et al. (2020). Interpretable off-policy evaluation in reinforcement learning by highlighting influential transitions. In: *ICML*, Proceedings of Machine Learning Research, vol 119. PMLR, pp 3658–3667
- Guidotti, R. (2024). Counterfactual explanations and how to find them: literature review and benchmarking. *Data Mining and Knowledge Discovery*, 38(5), 2770–2824. <https://doi.org/10.1007/S10618-022-00831-6>
- Guidotti, R., Monreale, A., Ruggieri, S., et al. (2019). A survey of methods for explaining black box models. *ACM Computing Surveys*, 51(5), 93:1–93:42. <https://doi.org/10.1145/3236009>
- Guo, W., Wu, X., Khan, U., et al. (2021). EDGE: explaining deep reinforcement learning policies. In: *NeurIPS*. Curran Associates, Inc.
- Hansen, N., & Ostermeier, A. (2001). Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2), 159–195. <https://doi.org/10.1162/106365601750190398>
- Heuillet, A., Couthouis, F., & Rodríguez, N. D. (2021). Explainability in deep reinforcement learning. *Knowledge-Based Systems*, 214, Article 106685. <https://doi.org/10.1016/J.KNOSYS.2020.106685>
- Hickling, T., Zenati, A., Aouf, N., et al. (2024). Explainability in deep reinforcement learning: A review into current methods and applications. *ACM Comput Surv* 56(5):125:1–125:35. <https://doi.org/10.1145/3623377>
- Huang, S. H., Bhatia, K., Abbeel, P., et al. (2018). Establishing appropriate trust via critical states. In: *IROS*. IEEE, pp 3929–3936, <https://doi.org/10.1109/IROS.2018.8593649>
- Kadir, M. A., Mosavi, A., & Sonntag, D. (2023). Evaluation metrics for xai: A review, taxonomy, and practical applications. In: *INES*. IEEE, pp 111–124, <https://doi.org/10.1109/INES59282.2023.10297629>
- Klemmt, A., Kutschke, J., & Schubert, C. (2017). From dispatching to scheduling: Challenges in integrating a generic optimization platform into semiconductor shop floor execution. In: *WSC*. IEEE, pp 3691–3702, <https://doi.org/10.1109/WSC.2017.8248081>
- Konda, V. R., & Tsitsiklis, J. N. (1999). Actor-critic algorithms. In: *NIPS*. The MIT Press, pp 1008–1014
- Kopp, D., Hassoun, M., Kalir, A., et al. (2020). Smt 2020a semiconductor manufacturing testbed. *IEEE Transactions on Semiconductor Manufacturing*, 33(4), 522–531. <https://doi.org/10.1109/TSM.2020.3001933>
- Kovács, B., Tassel, P., & Gebser, M. (2023). Optimizing dispatching strategies for semiconductor manufacturing facilities with genetic programming. In: *GECCO*. ACM, pp 1374–1382, <https://doi.org/10.1145/3583131.3590402>
- Ku, W., & Beck, J. C. (2016). Mixed integer programming models for job shop scheduling: A computational analysis. *Computers & Operations Research*, 73, 165–173. <https://doi.org/10.1016/J.COR.2016.04.006>

- Kuhnle, A., May, M. C., Schäfer, L., et al. (2022). Explainable reinforcement learning in production control of job shop manufacturing system. *International Journal of Production Research*, 60(19), 5812–5834. <https://doi.org/10.1080/00207543.2021.1972179>
- Lundberg, S. M., & Lee, S. (2017). A unified approach to interpreting model predictions. In: *NIPS*. Curran Associates, Inc., pp 4765–4774
- Milani, S., Topin, N., Veloso, M., et al. (2024). Explainable reinforcement learning: A survey and comparative review. *ACM Comput Surv* 56(7):168:1–168:36. <https://doi.org/10.1145/3616864>
- Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2013). Playing atari with deep reinforcement learning. *CoRR* abs/1312.5602
- Mnih, V., Badia, A. P., Mirza, M., et al. (2016). Asynchronous methods for deep reinforcement learning. In: *ICML, JMLR Workshop and Conference Proceedings*, vol 48. JMLR.org, pp 1928–1937
- Molnar, C. (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, 2nd edn.
- Mönch, L., Fowler, J. W., Dauzère-Pérès, S., et al. (2011). A survey of problems, solution techniques, and future challenges in scheduling semiconductor manufacturing operations. *Journal of scheduling*, 14(6), 583–599. <https://doi.org/10.1007/S10951-010-0222-9>
- Mönch, L., Fowler, J. W., & Mason, S. J. (2013). Production Planning and Control for Semiconductor Wafer Fabrication Facilities - Modeling, Analysis, and Systems, Operations research / computer science interfaces series, vol 52. Springer. <https://doi.org/10.1007/978-1-4614-4472-5>
- Mørch, N. J. S., Kjems, U., Hansen, L. K., et al. (1995). Visualization of neural networks using saliency maps. In: *ICNN*. IEEE, pp 2085–2090. <https://doi.org/10.1109/ICNN.1995.488997>
- Olson, M. L., Khanna, R., Neal, L., et al. (2021). Counterfactual state explanations for reinforcement learning agents via generative deep learning. *Artificial Intelligence*, 295, Article 103455. <https://doi.org/10.1016/J.ARTINT.2021.103455>
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *The Journal of machine Learning research*, 12, 2825–2830.
- Retzlaff, C. O., Angerschmid, A., Saranti, A., et al. (2024). Post-hoc vs ante-hoc explanations: xai design guidelines for data scientists. *Cognitive Systems Research*, 86, Article 101243. <https://doi.org/10.1016/J.COGLYSYS.2024.101243>
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “why should I trust you?”: Explaining the predictions of any classifier. In: *HLT-NAACL Demos*. The Association for Computational Linguistics, pp 97–101. <https://doi.org/10.18653/V1/N16-3020>
- Salimans, T., Ho, J., Chen, X., et al. (2017). Evolution strategies as a scalable alternative to reinforcement learning. *CoRR* abs/1703.03864
- Schulman, J., Wolski, F., Dhariwal, P., et al. (2017). Proximal policy optimization algorithms. *CoRR* abs/1707.06347
- Shapley, L. S. (1953). *17. A Value for n-Person Games*, Princeton University Press, pp 307–318. <https://doi.org/10.1515/9781400881970-018>
- Spier, J., & Kempf, K. (1995). Simulation of emergent behavior in manufacturing systems. In: *ASMC*. IEEE, pp 90–94. <https://doi.org/10.1109/ASMC.1995.484347>
- Staudacher, J., & Pollmann, T. (2023). Assessing antithetic sampling for approximating shapley, banzhaf, and owen values. *AppliedMath*, 3(4), 957–988. <https://doi.org/10.3390/appliedmath3040049>
- Stöckermann, P., Immordino, A., Altenmüller, T., et al. (2023). Dispatching in real frontend fabs with industrial grade discrete-event simulations by deep reinforcement learning with evolution strategies. In: *WSC*. IEEE, pp 3047–3058. <https://doi.org/10.1109/WSC60868.2023.10408625>
- Strumbelj, E., & Kononenko, I. (2014). Explaining prediction models and individual predictions with feature contributions. *Knowledge and information systems*, 41(3), 647–665. <https://doi.org/10.1007/S10115-013-0679-X>
- Stöckermann, P., Südfeld, H., Immordino, A., et al. (2025). Scalability of reinforcement learning methods for dispatching in semiconductor frontend fabs: A comparison of open-source models with real industry datasets. *CoRR* abs/2505.11135
- Sutton, R. S., & Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning Series: MIT Press.
- Tassel, P., Kovács, B., Gebser, M., et al. (2023). Semiconductor fab scheduling with self-supervised and reinforcement learning. In: *WSC*. IEEE, pp 1924–1935. <https://doi.org/10.1109/WSC60868.2023.10407747>
- Vaswani, A., Shazeer, N., Parmar, N., et al (2017). Attention is all you need. In: *NIPS*. Curran Associates, Inc., pp 5998–6008
- Vig, J. (2019). A multiscale visualization of attention in the transformer model. In: *ACL (3)*. Association for Computational Linguistics, pp 37–42. <https://doi.org/10.18653/V1/P19-3007>
- Xu, K., Ba, J., Kiros, R., et al. (2015). Show, attend and tell: Neural image caption generation with visual attention. In: *ICML, JMLR Workshop and Conference Proceedings*, vol 37. JMLR.org, pp 2048–2057

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.